

Шаблон символьного драйвера

Владислав Богданов

2026-06-16

```

#include <linux/module.h>
#include <linux/fs.h>
#include <linux/device.h>
#include <linux/uaccess.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Vlad");

// === ПЕРЕМЕННЫЕ ДРАЙВЕРА ===
static dev_t dev_num;           // Номер устройства (major + minor)
static struct cdev my_cdev;     // Структура символического устройства
static struct class *my_class;  // Класс устройства (для /sys/class/)
static struct device *my_device; // Устройство (для /dev/ и udev)

#define DEV_NAME "radio"

// === ФУНКЦИИ ФАЙЛОВЫХ ОПЕРАЦИЙ ===
static int radio_open(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "radio: device opened\n");
    return 0;
}

static int radio_release(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "radio: device closed\n");
    return 0;
}

static ssize_t radio_read(struct file *file, char __user *buf, size_t count, loff_t
*offset)
{
    printk(KERN_INFO "radio: read called\n");
    return 0; // EOF
}

static ssize_t radio_write(struct file *file, const char __user *buf, size_t count,
loff_t *offset)
{
    printk(KERN_INFO "radio: write called\n");
    return count;
}

// === ТАБЛИЦА ФАЙЛОВЫХ ОПЕРАЦИЙ ===
static struct file_operations fops = {
    .owner    = THIS_MODULE,
    .open     = radio_open,
    .release  = radio_release,
    .read     = radio_read,
    .write    = radio_write,

```

```

};

// === ИНИЦИАЛИЗАЦИЯ ===
static int __init radio_init(void)
{
    int ret;

    // 1. Запрашиваем диапазон номеров устройств (1 minor number)
    ret = alloc_chrdev_region(&dev_num, 0, 1, DEV_NAME);
    if (ret < 0) {
        printk(KERN_ERR "radio: cannot allocate chrdev region\n");
        return ret;
    }
    printk(KERN_INFO "radio: allocated major %d\n", MAJOR(dev_num));

    // 2. Инициализируем cdev и связываем с fops
    cdev_init(&my_cdev, &fops);
    my_cdev.owner = THIS_MODULE;

    // 3. Добавляем устройство в ядро
    ret = cdev_add(&my_cdev, dev_num, 1);
    if (ret < 0) {
        printk(KERN_ERR "radio: cdev_add failed\n");
        unregister_chrdev_region(dev_num, 1);
        return ret;
    }

    // 4. Создаем класс устройства (для ядер 6.4+)
    my_class = class_create(DEV_NAME);
    if (IS_ERR(my_class)) {
        printk(KERN_ERR "radio: class_create failed\n");
        cdev_del(&my_cdev);
        unregister_chrdev_region(dev_num, 1);
        return PTR_ERR(my_class);
    }

    // 5. Создаем устройство – udev автоматически создаст /dev/radio
    my_device = device_create(my_class, NULL, dev_num, NULL, DEV_NAME);
    if (IS_ERR(my_device)) {
        printk(KERN_ERR "radio: device_create failed\n");
        class_destroy(my_class);
        cdev_del(&my_cdev);
        unregister_chrdev_region(dev_num, 1);
        return PTR_ERR(my_device);
    }

    printk(KERN_INFO "radio: driver loaded, /dev/%s created\n", DEV_NAME);
    return 0;
}

// === ВЫГРУЗКА ===

```

```
static void __exit radio_exit(void)
{
    // Уничтожаем в ОБРАТНОМ порядке
    device_destroy(my_class, dev_num);
    class_destroy(my_class);
    cdev_del(&my_cdev);
    unregister_chrdev_region(dev_num, 1);

    printk(KERN_INFO "radio: driver unloaded\n");
}

module_init(radio_init);
module_exit(radio_exit);
```

Что нужно (чек-лист) Обязательно:

- `dev_t dev_num` — номер устройства
- `struct cdev my_cdev` — символьное устройство
- `struct class *my_class` — класс для sysfs
- `struct device *my_device` — устройство для `/dev/`
- `struct file_operations fops` — заполненный функциями
- `alloc_chrdev_region()` + `cdev_init()` + `cdev_add()` — регистрация
- `class_create()` + `device_create()` — создание `/dev/`
- `module_init()` + `module_exit()` — точки входа/выхода
- `init` + `exit` — макросы для оптимизации памяти

НЕ нужно:

- `register_chrdev()` — это legacy API
- `int radio_major` — используем `dev_t`
- `MKDEV()` — `dev_num` уже содержит `major+minor`
- Ручное создание `/dev/radio` через `mknod`

Тестирование

```
# Компиляция
make

# Загрузка модуля
sudo insmod radio.ko

# Проверка
ls -l /dev/radio      # Должен появиться
cat /dev/radio        # Вызовет radio_read()
echo "test" > /dev/radio # Вызовет radio_write()
```

```
# Просмотр логов  
dmesg | tail
```

```
# Выгрузка  
sudo rmmmod radio
```